Proyecto final Ingeniería de Software V: Circle Guard

Integrantes:

- Melissa Hurtado Becerra - A00401116
- Isabella Cuervo Vargas - A00401334
- Valentina Tobar Gomez - A00401749
- Santiago Grajales Perez - A00402018

Sprint 1

Contexto:
El Sprint 1 se desarrolló entre el 1 de noviembre y el domingo 8 de noviembre de 2026, con la entrega de la Parte 1 del proyecto. Su objetivo fue contar con un pipeline CI/CD inicial funcionando de punta a punta, que cubra el ciclo desarrollo, construcción, pruebas, empaquetamiento y despliegue. Para este sprint se comprometieron 20 de las 32 historias del backlog. Teniendo una estimación de 91 puntos de historia y una capacidad del equipo del total de los puntos.

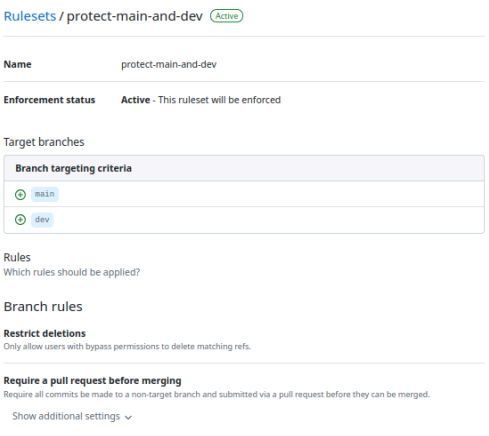
Repositorio del CircleGuard-Llibrary:
https://github.com/isabellacv1/circleguard-jenkins-library

Backlog en Jira:
https://circleguard-msvi.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog

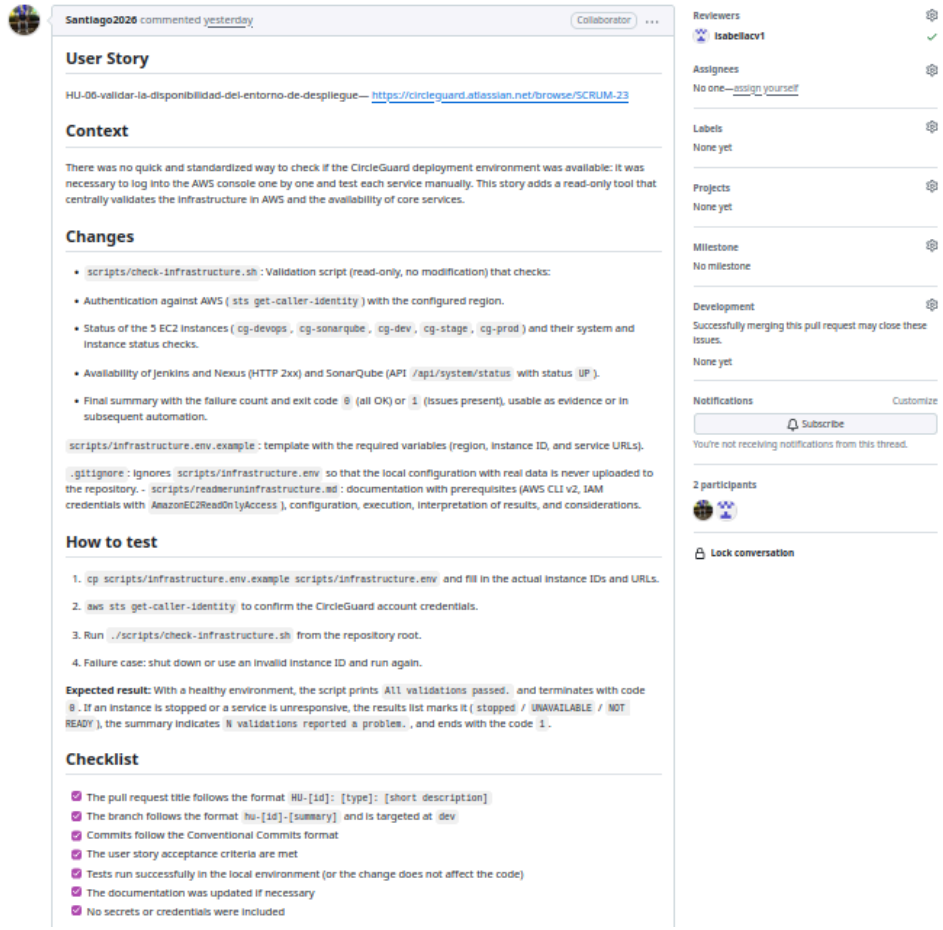
Ahí podrás encontrar el detalle de cada una de las historias de usuario con su descripción en el formato Como, Quiero y Para, así como podrás encontrar los criterios de aceptación, el responsable y su estimación y priorización.

Desarrollo del Sprint 1:

HU-001: Establecer las reglas de colaboración del repositorio
Se aplicaron en GitHub las reglas de protección de las ramas main y dev definidas en el Sprint 0. En ambas ramas se exige la creación de un pull request antes de fusionar, al menos una aprobación de un integrante distinto al autor, y se bloquean el push directo, el force push y la eliminación de la rama. También se configuraron como obligatorios los checks del pipeline de Jenkins (continuous-integration/jenkins/branch y continuous-integration/jenkins/pr-merge), de modo que un pull request no puede fusionarse si la construcción o las pruebas fallan.



HU-002: Estandarizar el flujo de pull requests
Se definió la plantilla de pull request en .github/pull_request_template.md. La plantilla incluye las secciones User story, Context, Changes, How to test y Checklist, y recuerda el formato de título HU-[id]: [tipo]: [descripción breve]. También indica que la rama debe seguir hu-[id]-[resumen], que el PR debe apuntar a dev, que los commits deben seguir Conventional Commits y que se requiere al menos una aprobación de otro integrante.



Santiago2026 commented yesterday — Collaborator

User Story

HU-06-validar-la-disponibilidad-del-entorno-de-despliegue— https://circleguard.atlassian.net/browse/SCRUM-23

Context

There was no quick and standardized way to check if the CircleGuard deployment environment was available: it was necessary to log into the AWS console one by one and test each service manually. This story adds a read-only tool that centrally validates the infrastructure in AWS and the availability of core services.

Changes

- `scripts/check-infrastructure.sh` : Validation script (read-only, no modification) that checks:
- Authentication against AWS (`sts get-caller-identity`) with the configured region.
- Status of the 5 EC2 instances (`cg-devops` , `cg-sonarqube` , `cg-dev` , `cg-stage` , `cg-prod`) and their system and instance status checks.
- Availability of Jenkins and Nexus (HTTP 2xx) and SonarQube (API `/api/system/status` with status `UP`).
- Final summary with the failure count and exit code `0` (all OK) or `1` (issues present), usable as evidence or in subsequent automation.

`scripts/infrastructure.env.example` : template with the required variables (region, instance ID, and service URLs).

`.gitignore` : Ignores `scripts/infrastructure.env` so that the local configuration with real data is never uploaded to the repository. - `scripts/readmeruninfrastructure.md` : documentation with prerequisites (AWS CLI v2, IAM credentials with `AmazonEC2ReadOnlyAccess`), configuration, execution, interpretation of results, and considerations.

How to test

1. `cp scripts/infrastructure.env.example scripts/infrastructure.env` and fill in the actual instance IDs and URLs.
2. `aws sts get-caller-identity` to confirm the CircleGuard account credentials.
3. Run `./scripts/check-infrastructure.sh` from the repository root.
4. Failure case: shut down or use an invalid instance ID and run again.

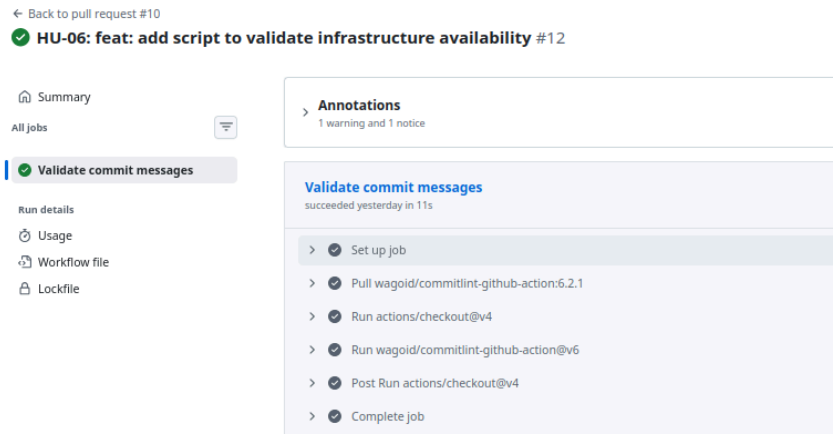
Expected result: With a healthy environment, the script prints `All validations passed.` and terminates with code `0` . If an instance is stopped or a service is unresponsive, the results list marks it (`stopped` / `UNAVAILABLE` / `NOT READY`), the summary indicates `N validations reported a problem.` , and ends with the code `1` .

Checklist

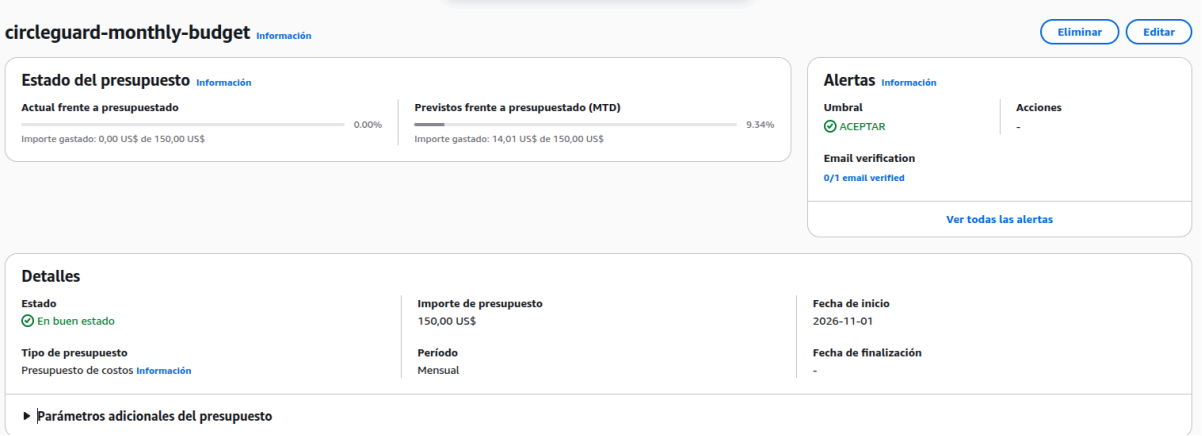
- [x] The pull request title follows the format `HU-[id]: [type]: [short description]`
- [x] The branch follows the format `hu-[id]-[summary]` and is targeted at `dev`
- [x] Commits follow the Conventional Commits format
- [x] The user story acceptance criteria are met
- [x] Tests run successfully in the local environment (or the change does not affect the code)
- [x] The documentation was updated if necessary
- [x] No secrets or credentials were included

Reviewers: isabellacv1
Assignees: No one—assign yourself
Labels: None yet
Projects: None yet
Milestone: No milestone
Development: Successfully merging this pull request may close these issues. None yet
Notifications: Customize — Subscribe — You're not receiving notifications from this thread.
2 participants
Lock conversation

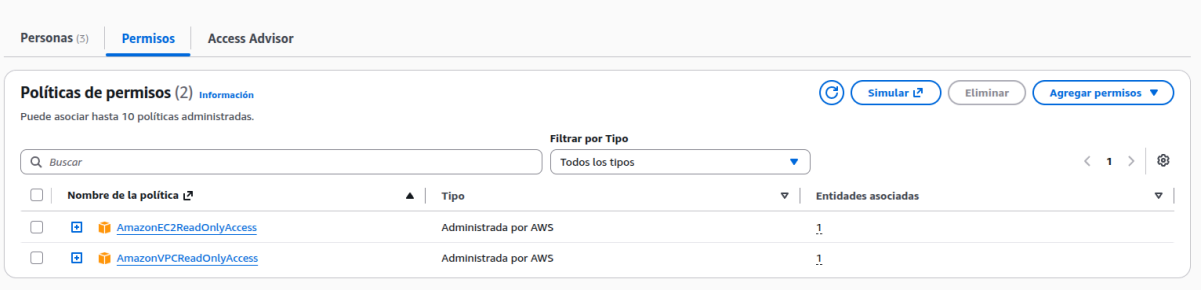
HU-003: Estandarizar los mensajes de commit
Se adoptó Conventional Commits y se configuró commitlint en commitlint.config.mjs. Los tipos permitidos son feat, fix, perf, refactor, docs, test, build, ci, chore, style y revert. Los scopes recomendados incluyen los ocho servicios (auth, identity, promotion, form, gateway, notification, file, dashboard), el cliente (mobile) y ámbitos de plataforma (ci, infra, deploy, k8s, observability, security, docs, deps, tests). El asunto debe ir en minúsculas, sin punto final, y el encabezado puede tener hasta 100 caracteres. La validación se hace en dos lugares: localmente con el hook .githooks/commit-msg, que se activa con git config core.hooksPath .githooks, y en cada pull request con el workflow Commit lint de GitHub Actions (.github/workflows/commitlint.yml), que ejecuta commitlint sobre todos los commits del PR. La convención completa está documentada en docs/CONTRIBUTING.md.



HU-004 Preparar entorno de la nube:
Lo primero que hicimos fue generar un presupuesto para el proyecto, el cual tiene un presupuesto de 150 dólares y además generamos 4 alertas una cuando el presupuesto llegue al 50%, 75%, 90% y 100%



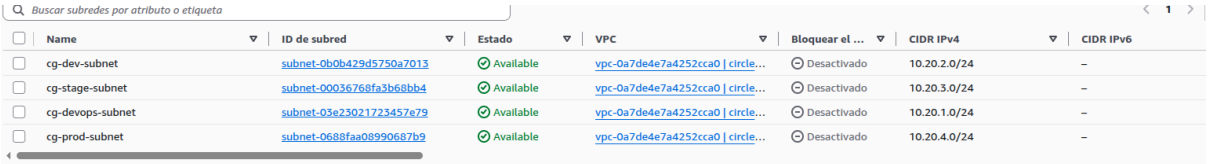
Establecimos un esquema de control de acceso para la infraestructura de CircleGuard en AWS, garantizando que cada integrante del equipo tenga únicamente los permisos necesarios para realizar sus actividades.



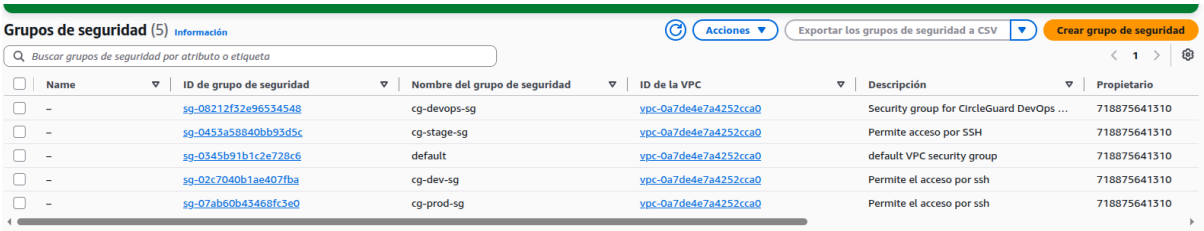
Se creó una infraestructura de red independiente para CircleGuard utilizando Amazon Virtual Private Cloud (VPC), con el propósito de organizar y controlar la conectividad de los recursos necesarios para el despliegue del proyecto.

Se crearon cuatro subredes dentro de la VPC, destinadas a separar los componentes de infraestructura compartida y los ambientes de ejecución:

- cg-devops-subnet: 10.20.1.0/24.
- cg-dev-subnet: 10.20.2.0/24.
- cg-stage-subnet: 10.20.3.0/24.
- cg-prod-subnet: 10.20.4.0/24.

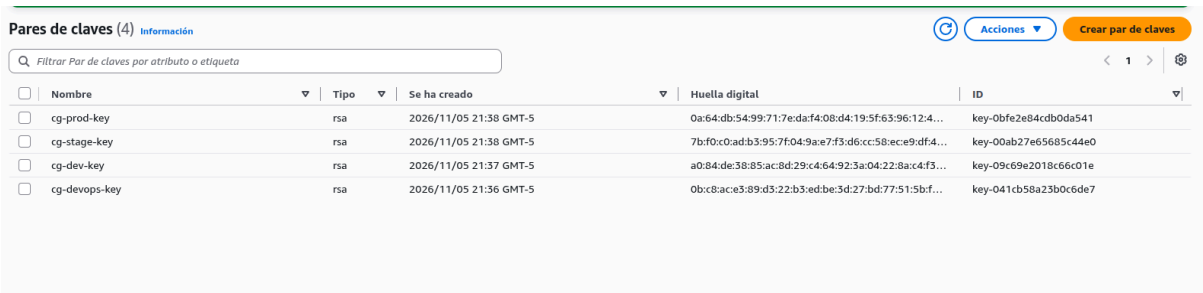


Definimos las reglas de tráfico de red que controlan las conexiones entrantes y salientes de las instancias EC2 de CircleGuard, limitando el acceso a los servicios según las responsabilidades de cada servidor y reduciendo la exposición de la infraestructura.

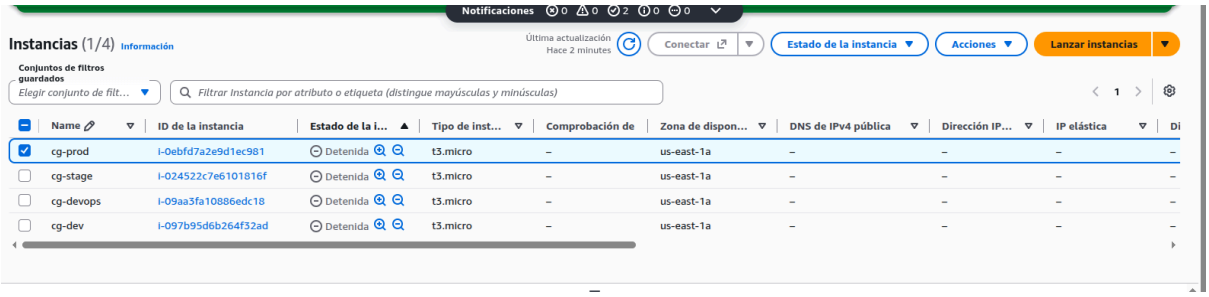


Configuramos los mecanismos de autenticación necesarios para establecer conexiones SSH seguras con las futuras instancias EC2 del proyecto CircleGuard.

Se utilizarán pares de claves SSH. Cada ambiente tendrá su propio par de claves, permitiendo administrar el acceso de forma independiente y reducir el impacto ante una posible exposición de una clave.



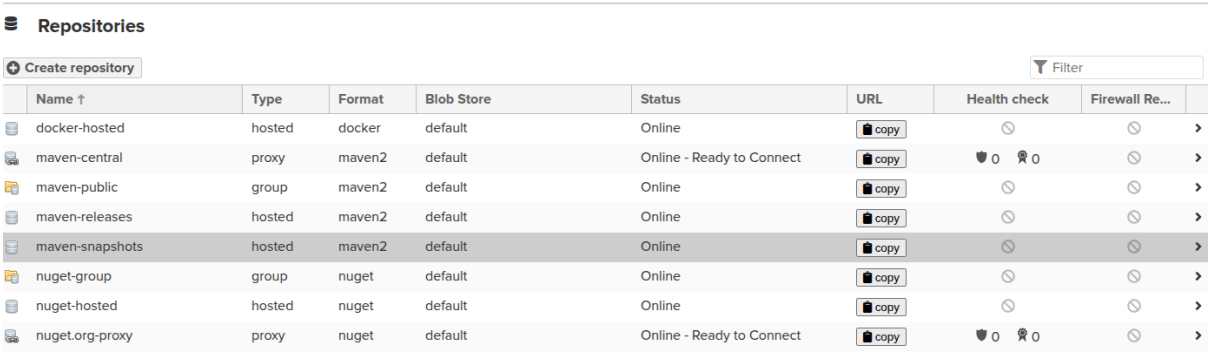
Se completó la creación de las cinco instancias EC2 de AWS para la infraestructura de CircleGuard: cg-devops, cg-dev, cg-stage y cg-prod y cg-sonarqube. Cada instancia se asoció a la VPC circleguard-vpc, utilizando su respectiva subnet, clave SSH (cg-devops-key, cg-dev-key, cg-stage-key y cg-prod-key) y grupo de seguridad (cg-devops-sg, cg-dev-sg, cg-stage-sg y cg-prod-sg). Además, se configuró almacenamiento EBS gp3 cifrado, con 100 GB para DevOps y 50 GB para cada ambiente, dejando



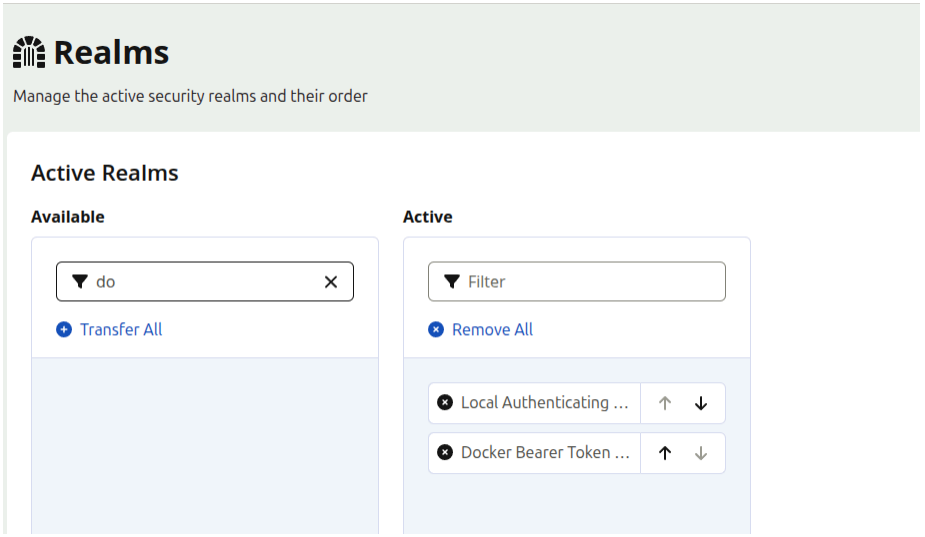
HU-005-Automatizar tareas de infraestructura mediante Bash
Empezamos instalando Docker en todas las instancias mediante un script que se encuentra en: scripts/install-docker.sh reutilizado de los talleres desarrollados en clase

Vamos a empezar con el aprovisionamiento de las 3 herramientas en las instancias que decidimos que tendrían este rol.

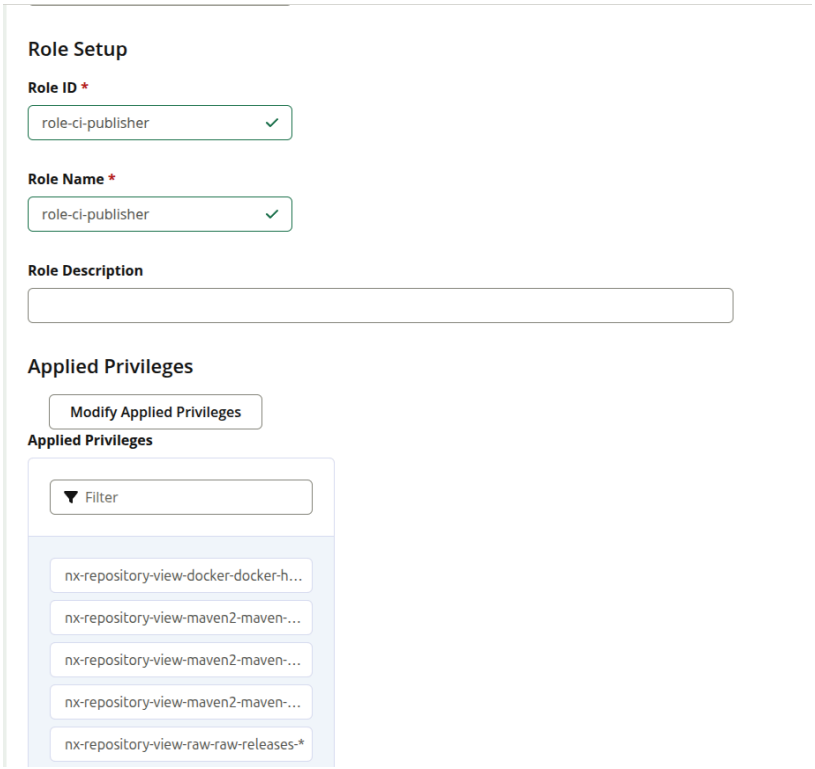
- **Nexus:** Primero por medio de scp enviamos la configuración de nexus con un docker compose, .env y un archivo sh para solo ejecutar este y que arranque el contenedor de nexus y le muestre la contraseña una vez encendido el contenedor nos conectamos desde nuestra máquina con la <ip-publica-instancia>:8081 y realizamos la siguiente configuración:
    - Creamos docker-hosted para nuestras imágenes:



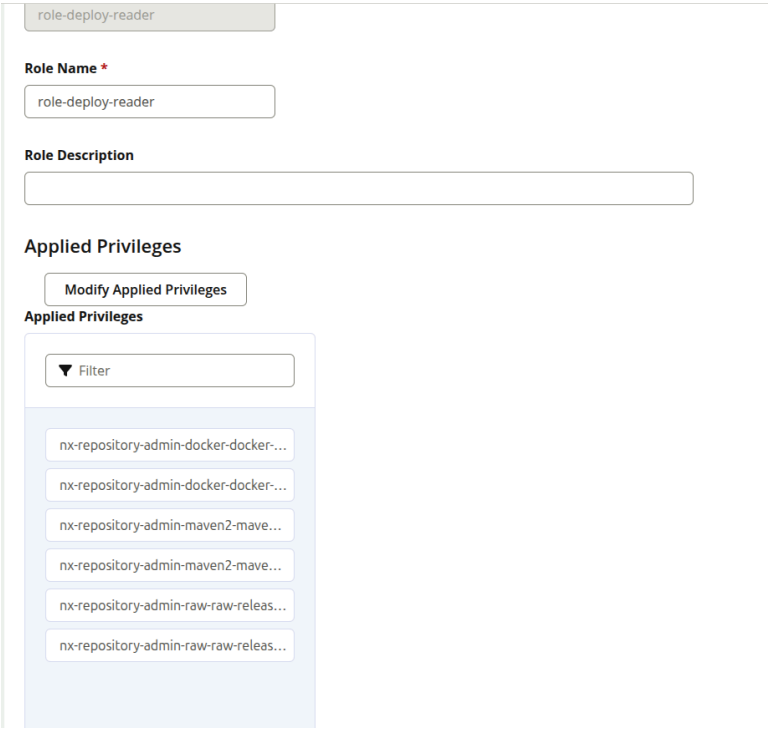
Activamos el Realm: Security → Realms → Docker Bearer Token Realm → Active.



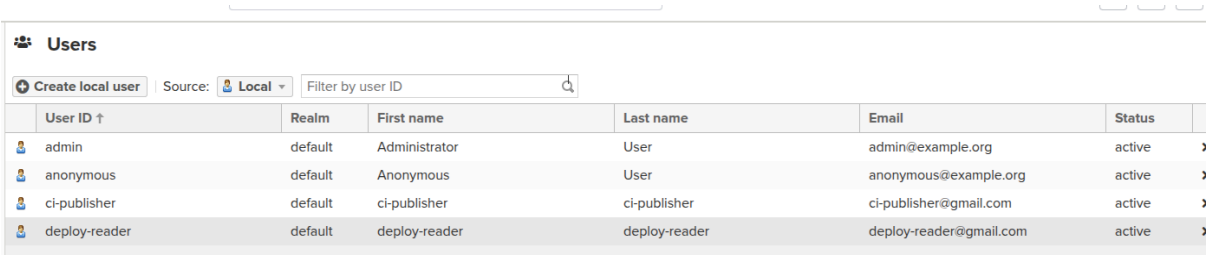
Se creó el rol :role-ci-publisher con los siguientes permisos:



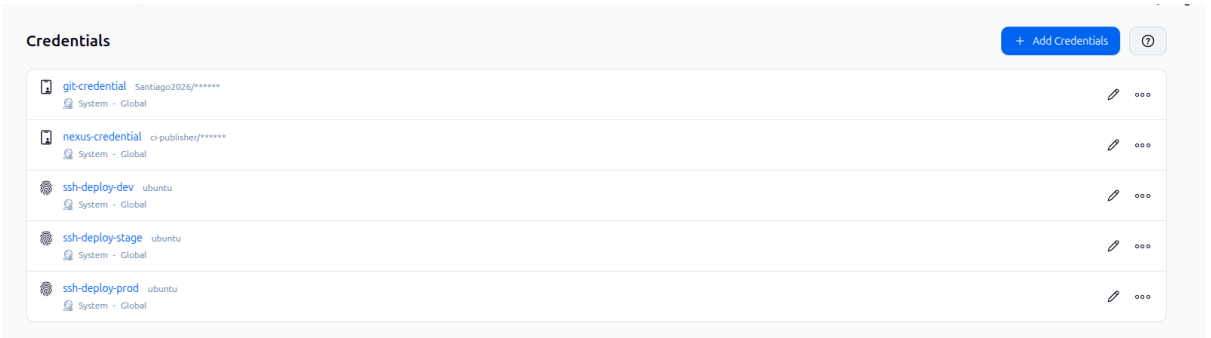
y el rol role-deploy-reader: read con los siguientes permisos:



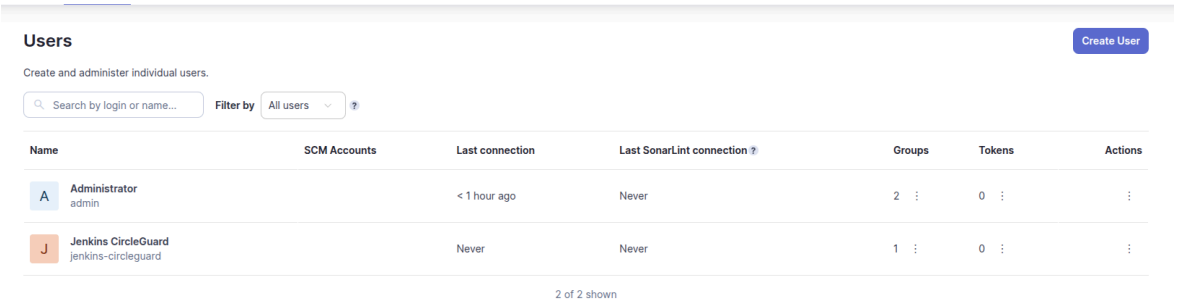
Además se crearon los usuarios con sus respectivos roles, primero creamos ci-publisher y deploy-reader:



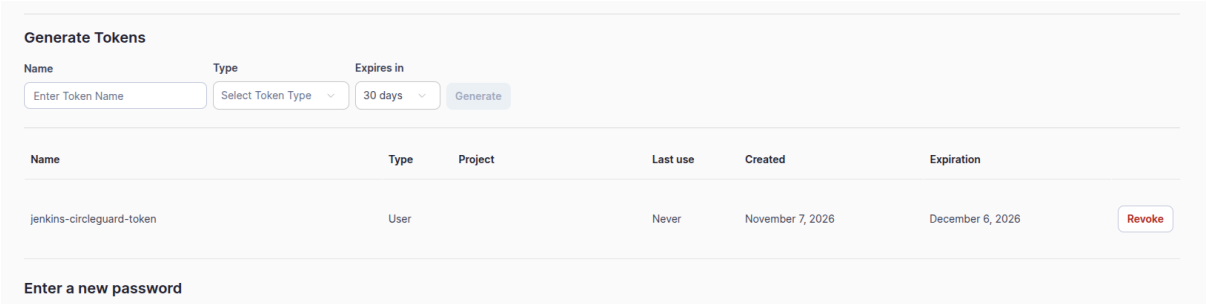
Jenkins: primero por medio de scp enviamos la configuración de Jenkins con un docker compose, .env, dockerfile, una carpeta smee con su dockerfiles y un archivo sh para solo ejecutar este y que arranque el contenedor de jenkins y le muestre la contraseña una vez encendido el contenedor nos conectamos desde nuestra máquina con la <ip-publica-instancia>:8080 y realizamos la siguiente configuración:
Credenciales iniciales:



Sonarqube: Realizamos lo mismo que realizamos para nexus y jenkins y creamos un usuario:



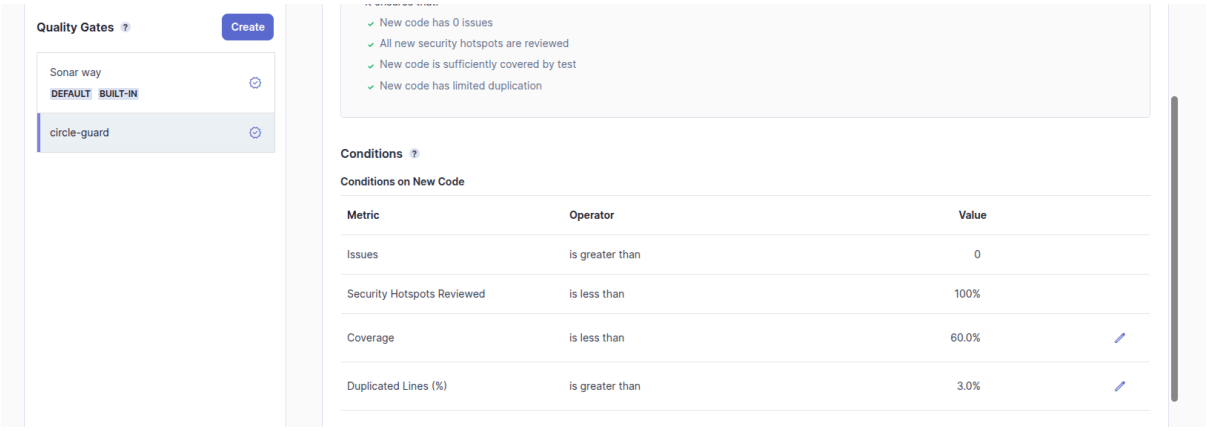
Generamos un token para poder que jenkins se pueda conectar a sonarqube:



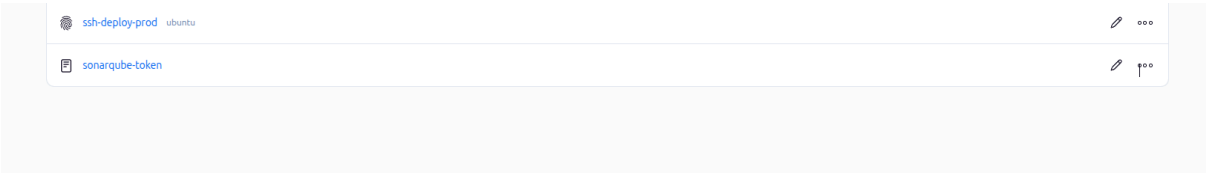
Creamos un proyecto en sonarqube:



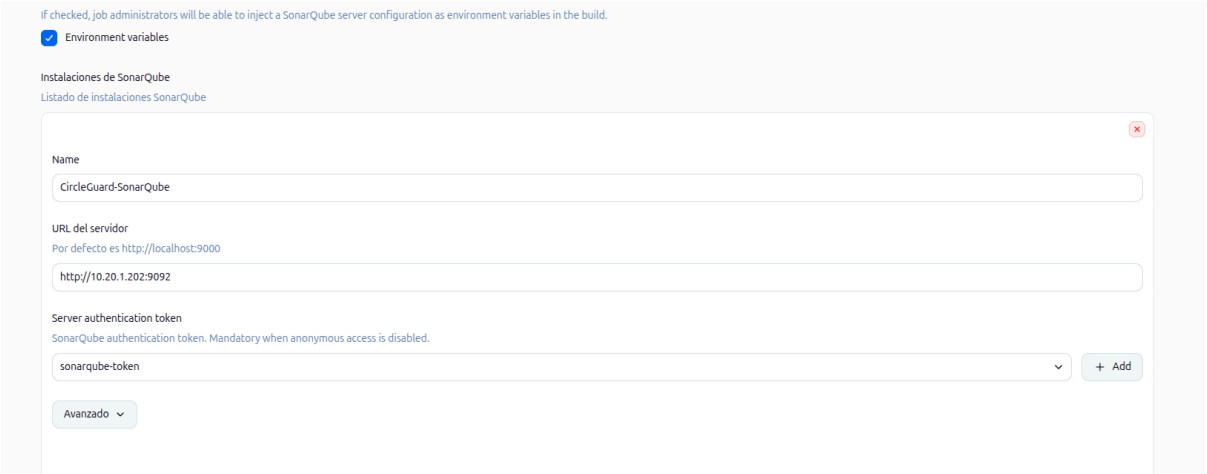
Creamos quality gates para el proyecto:



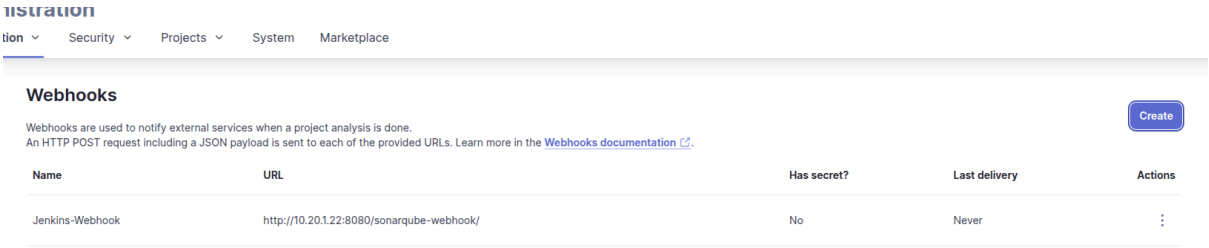
y lo asociamos al proyecto
ahora añadimos la credencial de sonarqube a jenkins:



Registramos el servidor de sonarqube en jenkins:



Configuramos el webhook en sonarqube:



HU-006: Validar la disponibilidad del entorno de despliegue

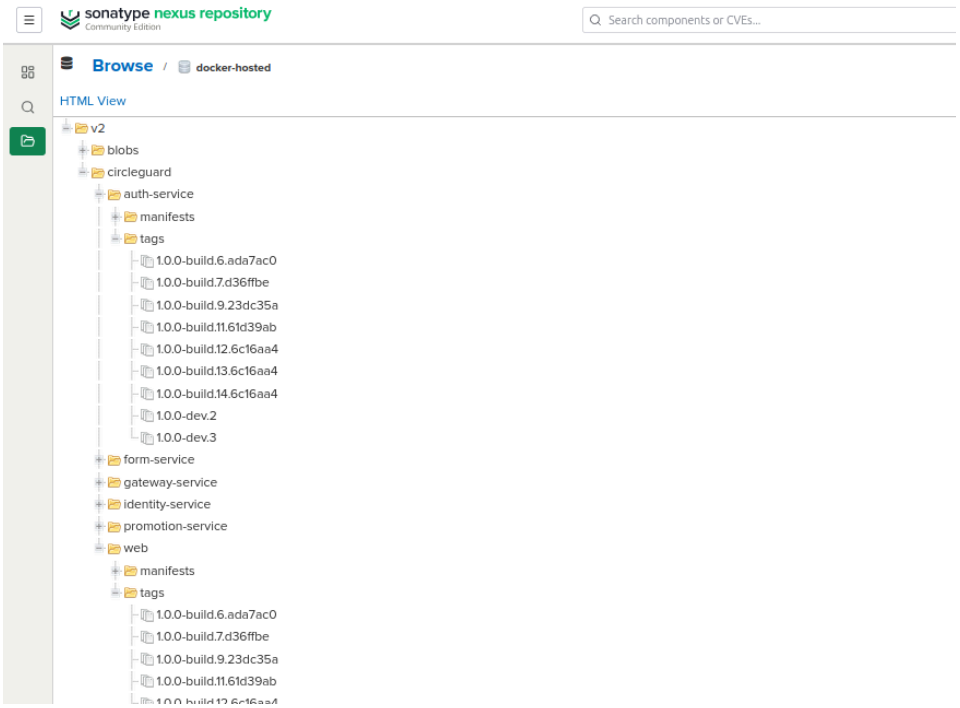
Se validó la disponibilidad del entorno de despliegue con scripts/check-infrastructure.sh, que consulta el estado de las cinco instancias EC2 (cg-devops, cg-sonarqube, cg-dev, cg-stage y cg-prod) y la disponibilidad de Jenkins (/login), Nexus (/) y SonarQube (/api/system/status). El script requiere scripts/infrastructure.env con los IDs de instancia y las URLs, usa AWS CLI con permisos de solo lectura y devuelve código 0 si todas las validaciones pasan. La ejecución reportó las cinco instancias en estado running con System: OK e Instance: OK, y los tres servicios disponibles: Jenkins y Nexus con HTTP 2xx y SonarQube con estado UP. Adicionalmente se verificó el acceso SSH a cada instancia, que Docker y Docker Compose respondieran, y que hubiera memoria y disco suficientes para el despliegue.

HU-007: Crear imágenes Docker de los servicios principales

Se construyeron Dockerfiles multi-stage para los cinco servicios principales:

- circleguard-auth-service (8180)
- circleguard-identity-service (8083)
- circleguard-promotion-service (8088)
- circleguard-form-service (8086)
- circleguard-gateway-service (8087).

Cada Dockerfile usa eclipse-temurin:21.0.8_9-jdk-jammy en la etapa de construcción y eclipse-temurin:21.0.8_9-jre-jammy en la etapa de ejecución, compila solo el módulo correspondiente con Gradle y copia el JAR a /app/app.jar. La imagen final crea el usuario de sistema app, ejecuta el proceso con USER app (no root), declara el puerto con EXPOSE y define un HEALTHCHECK que abre una conexión TCP contra el puerto del servicio. Las imágenes se etiquetan con la versión semántica resuelta por el pipeline (resolveVersion) y se publican en Nexus con docker build + docker push; nunca se usa latest.



HU-008: Contenerizar la aplicación web
Se creó una imagen multi-stage para el cliente Expo. Para la etapa de construcción usa node:22.21.1-alpine3.22, instala dependencias con npm ci y genera la salida web con npx expo export --platform web --output-dir dist. La etapa final usa nginxinc/nginx-unprivileged:1.28.0-alpine3.21, que ejecuta nginx como usuario no root y escucha en el puerto 8080; el host publica el puerto 8081. Las URLs de los servicios no se hardcodean en la imagen, además se inicia el contenedor a partir de las variables AUTH_BASE_URL, IDENTITY_BASE_URL, FORM_BASE_URL, GATEWAY_BASE_URL y PROMOTION_BASE_URL. En docker-compose.app.yml y deploy/docker-compose.yml esas variables se pasan con localhost o con WEB_HOST, por lo que la misma imagen funciona en local y en cada ambiente. El healthcheck usa wget --spider contra http://127.0.0.1:8080/.

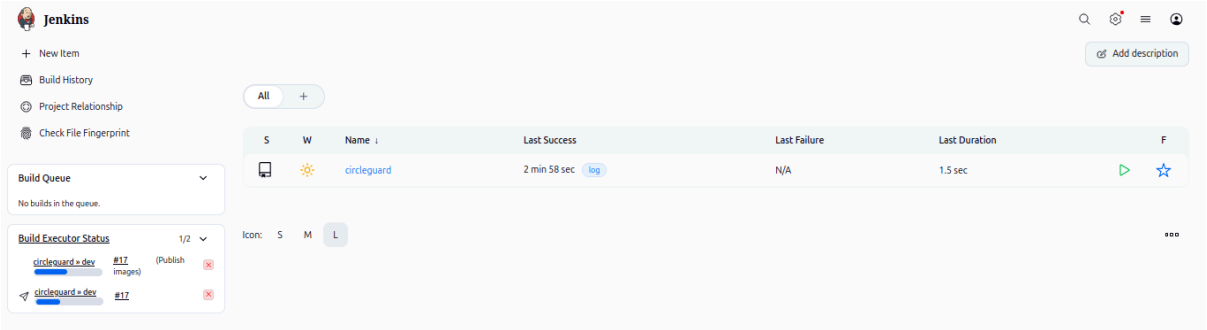
HU-009: Configurar la ejecución de CircleGuard mediante Docker Compose
Se configuró la ejecución completa de CircleGuard con Docker Compose. Donde docker-compose.dev.yml define la infraestructura con PostgreSQL 16, Neo4j 5.26, Redis 7.2, Kafka 7.6, todos con healthchecks y en la red circleguard-net. Se definen los servicios con app web y depends_on condicionado a service_healthy para arrancar solo cuando las dependencias están listas. Los datos de PostgreSQL y Neo4j se conservan en los

volúmenes pgdata y neo4jdata. La app web queda en http://localhost:8081 y los servicios en sus puertos: auth 8180, identity 8083, form 8086, gateway 8087 y promotion 8088.

HU-010: Configurar Jenkins como herramienta de integración continua

Se configuró Jenkins como herramienta de integración continua en la instancia cg-devops, desplegado con Docker Compose (infra/bootstrap/jenkins). Jenkins escucha en el puerto 8080 y usa el socket de Docker del host para ejecutar builds y contenedores de Testcontainers.

Se instalaron los plugins declarados en plugins.txt y se configuró un cliente de Smee (smee-client) para recibir los webhooks de GitHub, ya que Jenkins no tiene un endpoint HTTPS público. Se creó el job multibranch circleguard, definido en infra/jenkins/jobs/circleguard.xml, que escanea las ramas y los pull requests del repositorio y ejecuta el Jenkinsfile. Las credenciales necesarias (git-credential, nexus-credential, sonarqube-token, discord-webhook-url, ssh-deploy-dev, ssh-deploy-stage, ssh-deploy-prod, e2e-credential y e2e-credentials) se administran en Jenkins.



HU-011: Implementar el pipeline de construcción y pruebas

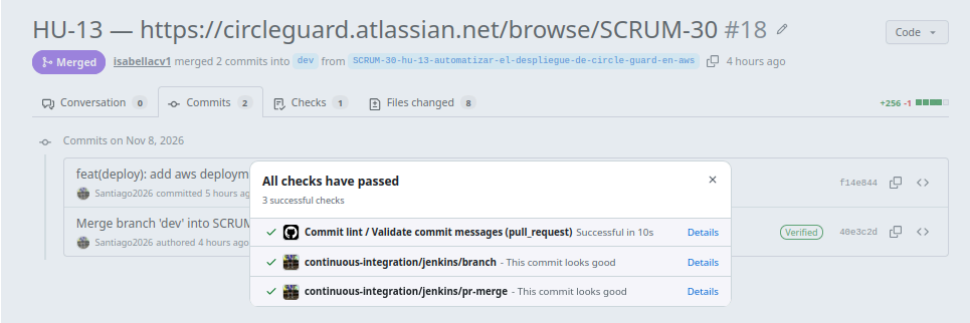
Se implementó el pipeline de construcción y pruebas con una librería compartida (circleguard-pipeline-library). El Jenkinsfile del repositorio solo carga la librería y pasa los parámetros: integrationTests: true, sonar: true, sonarProjectKey: 'CircleGuard', publish: true, deploy: true, environment: 'dev', e2e: true, promote: true y discordCredentialsId: 'discord-webhook-url'. El pipeline ejecuta las etapas Checkout, Build, Tests, Integration tests, SonarQube, Publish images, Deploy, Verify deployment, E2E tests, Promote to stage y Promote to prod. Las etapas posteriores solo se ejecutan si las anteriores terminan correctamente.



HU-012: Integrar las validaciones de CI con los pull requests

Se integraron las validaciones de CI con los pull requests. GitHub envía los eventos push y pull_request al canal de Smee, que los reenvía a Jenkins en /github-webhook/. El job multibranch circleguard construye las ramas y los PR del mismo repositorio; los PR se construyen fusionados con su rama destino. Jenkins reporta los estados continuous-integration/jenkins/branch y continuous-integration/jenkins/pr-merge a GitHub. En la protección de ramas dev y main se configuraron estos checks como obligatorios, por

lo que un PR no se puede fusionar si el pipeline falla. También se ejecuta el workflow Commit lint sobre los commits del PR.

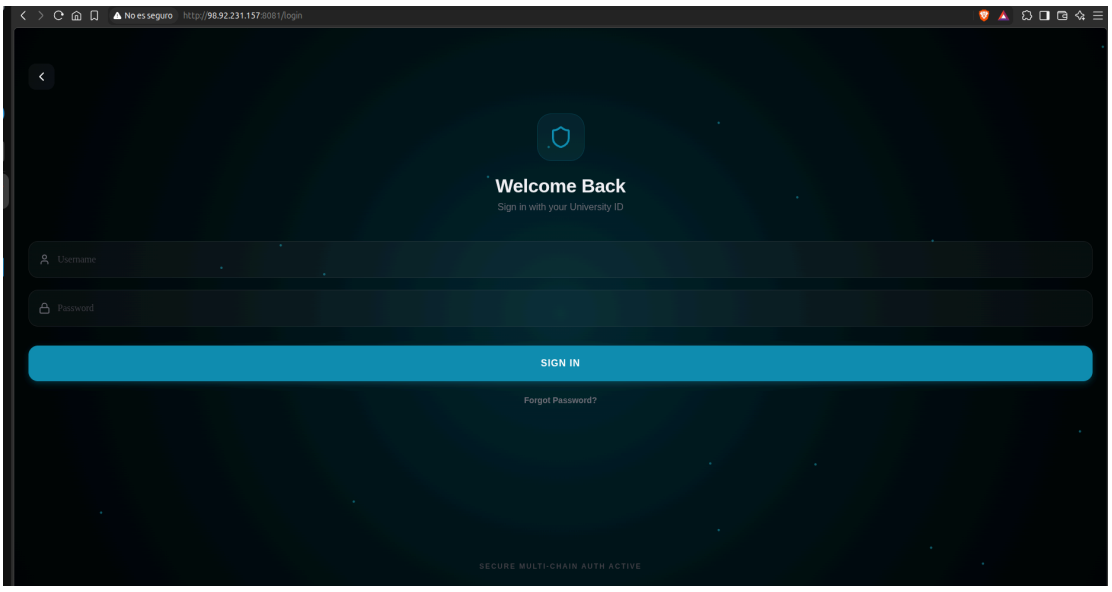


HU-013: Automatizar el despliegue de CircleGuard en AWS

Se automatizó el despliegue de CircleGuard en AWS. La etapa Deploy del pipeline usa el paso deployEnvironment de la librería, que lee deploy/environments/dev.env, se conecta por SSH a la instancia cg-dev con la credencial dev-credential, copia docker-compose.dev.yml, init-db.sql, deploy/docker-compose.yml y un archivo .env con REGISTRY y VERSION, y ejecuta docker compose pull y docker compose up -d --wait --wait-timeout 600. La versión desplegada es exactamente env.IMAGE_VERSION, la misma que publicó este build; nunca se usa latest. Después del despliegue, la etapa Verify deployment comprueba por HTTP que los seis componentes respondan.

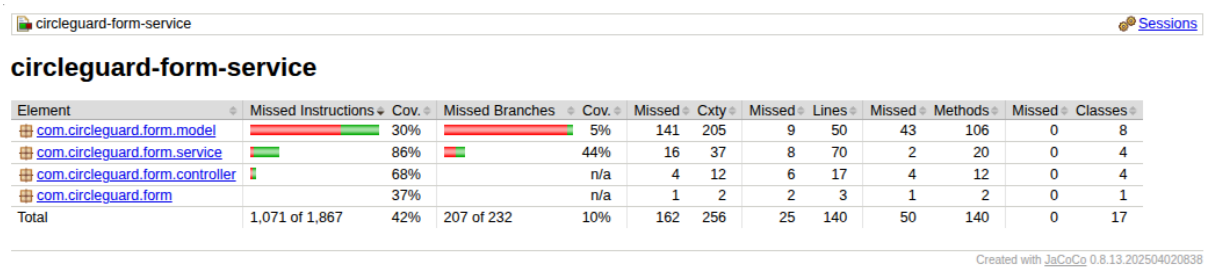
HU-014: Implementar la promoción entre ambientes

Se implementó la promoción entre ambientes con el paso promoteEnvironment de la librería. El flujo es dev -> stage -> prod. La promoción a stage es automática y solo se ejecuta si el build, las pruebas unitarias, las pruebas de integración, el despliegue en devy las pruebas E2E terminaron correctamente. La promoción a prod siempre requiere una aprobación manual mediante input, con un tiempo máximo de 240 minutos. Cada promoción despliega la misma versión (env.IMAGE_VERSION) que se publicó en el build y registra una entrada en promotion-record.txt con build, versión, ruta, condición, aprobador y fecha; el archivo se guarda como artefacto del build.



HU-015: Implementar validaciones posteriores al despliegue
Se implementaron validaciones posteriores al despliegue. La etapa Verify deployment usa verifyServices, que hace peticiones HTTP a los seis componentes (gateway 8087, auth 8180, identity 8083, form 8086, promotion 8088 y web 8081) con 20 reintentos cada 10 segundos y falla si alguno no responde por debajo de 500. La etapa Smoke tests ejecuta smokeTests, que corre comprobaciones funcionales en el host destino: página web, configuración runtime, validación de token en gateway, encuestas pendientes en form, estadísticas de promotion, identidad sin autenticación y login válido/inválido en auth. La etapa E2E tests ejecuta Playwright contra dev y es la condición para promover a stage. Los resultados se publican como reportes JUnit y artefactos del build.

HU-016: Incorporar pruebas automatizadas al proceso de CI
Se incorporaron pruebas automatizadas al proceso de CI. El pipeline ejecuta las pruebas unitarias de todos los servicios con ./gradlew test --continue y las pruebas de integración con ./gradlew integrationTest --continue; ambas usan Testcontainers. Se genera cobertura con JaCoCo y se publican los reportes JUnit y JaCoCo como artefactos. Las pruebas E2E con Playwright se ejecutan contra el ambiente dev desplegado y también se publican como reporte JUnit y HTML.


circleguard-form-service

Sessions

circleguard-form-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.circleguard.form.model		30%		5%	141	205	9	50	43	106	0	8
com.circleguard.form.service		86%		44%	16	37	8	70	2	20	0	4
com.circleguard.form.controller		68%		n/a	4	12	6	17	4	12	0	4
com.circleguard.form		37%		n/a	1	2	2	3	1	2	0	1
Total	1,071 of 1,867	42%	207 of 232	10%	162	256	25	140	50	140	0	17

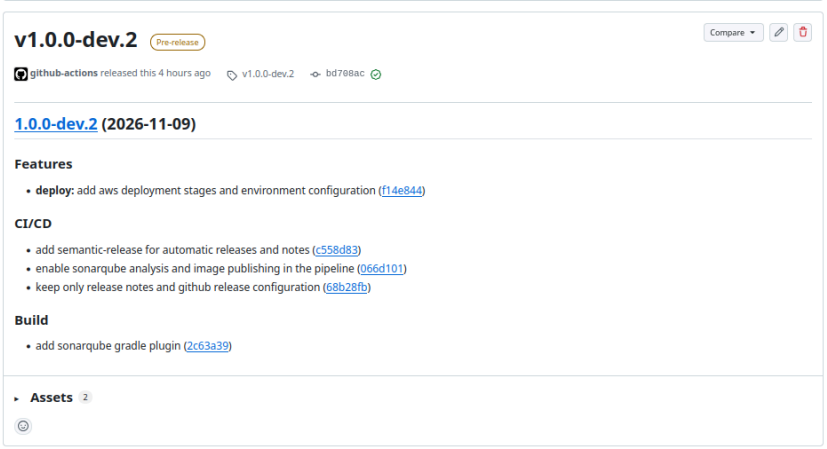
Created with JaCoCo 0.8.13.202504020838

HU-017: Validar la integración entre los componentes desplegados
Se validó la integración entre los componentes desplegados. verifyServices comprueba que los seis componentes estén disponibles por HTTP. smokeTests valida flujos funcionales que cruzan servicios: el login en auth pasa por LDAP, identity y PostgreSQL; la validación de token en gateway usa Redis; form y promotion responden con datos reales; y la web sirve la configuración runtime. Las pruebas E2E de Playwright recorren la aplicación web contra dev e incluyen autenticación y flujos de usuario. El pipeline solo promueve a stage si todas estas validaciones pasan.

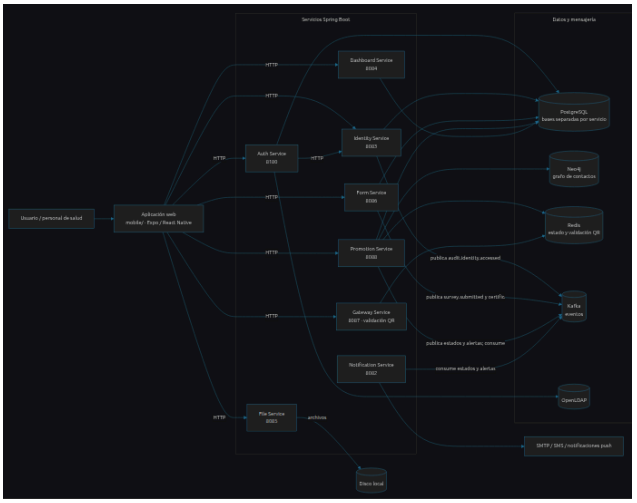
HU-018: Automatizar el versionado semántico
Se automatizó el versionado semántico con semantic-release. La configuración está en .releaserc.json: las ramas son main y dev (con prerelease dev), el formato de tag es v${version} y las reglas de release son feat → minor, fix/perf/revert → patch y breaking change → major. El workflow de GitHub Actions .github/workflows/semantic-versioning.yml se ejecuta en push a main y dev, calcula la versión a partir de los commits convencionales y crea el tag.



HU-019: Generar releases y notas de publicación automáticamente
Se automatizó la generación de releases y notas de publicación. semantic-release usa @semantic-release/release-notes-generator con el preset conventionalcommits para agrupar los cambios en secciones (Features, Bug Fixes, Performance, Reverts, Refactoring, Tests, CI/CD, Build, Documentation) y @semantic-release/github para publicar la GitHub Release asociada al tag. Cada release incluye las notas generadas a partir de los commits y queda disponible en la página de Releases del repositorio.



HU-020: Documentar el proceso de construcción y despliegue
Se documentó el proceso de construcción y despliegue. El repositorio incluye docs/README.md como punto de entrada técnico, con arquitectura, stack, ejecución local, reproducción del entorno completo, pruebas y CI/CD. La documentación se organiza en docs/arquitectura (diagrama y contexto técnico), docs/operaciones (pipeline, despliegue, versionado, release process, pruebas e infraestructura), docs/CONTRIBUTING.md (commits y PRs) y docs/agil (Sprint 0 y Sprint 1). Los documentos incluyen diagramas de arquitectura y pipeline, comandos reproducibles y enlaces a Jira y a la librería de Jenkins.



Sprint review: Durante la revisión del Sprint 1 se evaluó el cumplimiento de las 20 historias de usuario comprometidas y su contribución a la implementación de las prácticas DevOps en CircleGuard. Se verificaron las principales actividades:

- Gestión del flujo de trabajo: definición de reglas de colaboración, convenciones de commits y uso de pull requests.
- Infraestructura y contenerización: configuración de AWS, automatización mediante Bash, construcción de imágenes Docker y ejecución con Docker Compose.
- Integración continua: configuración de Jenkins y desarrollo de un pipeline para construcción, pruebas y análisis de calidad con SonarQube.
- Despliegue continuo: publicación de imágenes en Nexus, despliegues en AWS y promoción entre ambientes.
- Validación y versionado: incorporación de pruebas automatizadas, de integración y end-to-end, versionado semántico y releases en GitHub.

Como resultado, se obtuvo un flujo CI/CD funcional que automatiza la construcción, validación, publicación y despliegue de los servicios en dev, además de su promoción a stage. El despliegue en prod requiere aprobación manual. El proceso se respaldó con evidencias de ejecuciones en Jenkins, imágenes publicadas, reportes de pruebas y releases en GitHub.

En conclusión, el Sprint 1 permitió implementar un proceso DevOps integrado y automatizado, pasando de la preparación inicial a la ejecución de los principales procesos de CI/CD.

Sprint Retrospective: Al finalizar el Sprint 1, el equipo realizó una reflexión sobre el desarrollo de las actividades de implementación, identificando los aspectos positivos y aquellos que pueden mejorarse para el siguiente sprint.

¿Qué hicimos bien?

- Consolidamos una librería compartida de Jenkins que permitió reutilizar y organizar las etapas del pipeline.
- Implementamos un flujo CI/CD que integra construcción, pruebas, análisis de calidad, publicación de imágenes, despliegue y promoción entre ambientes.
- Incorporamos diferentes niveles de pruebas automatizadas para validar el funcionamiento de los servicios y su integración.
- Documentamos aspectos importantes de la arquitectura, el pipeline y el proceso de despliegue.
- Mantuvimos la colaboración entre los integrantes y el seguimiento de las actividades mediante Jira.

¿Qué podemos mejorar?

- Mejorar la estimación de las historias de usuario para distribuir mejor las responsabilidades y los tiempos de trabajo.
- Fortalecer la comunicación entre los integrantes para coordinar las actividades que dependen de diferentes componentes.
- Realizar validaciones más frecuentes durante la implementación para identificar errores antes de las etapas finales.

- Mantener la documentación y las evidencias actualizadas a medida que se completan las historias de usuario.
- Dedicar tiempo a revisar los resultados de las herramientas implementadas y detectar oportunidades de optimización.

Acciones de mejora para el siguiente sprint

- Revisar y ajustar el backlog y los criterios de aceptación antes de iniciar la implementación de las historias de usuario.
- Estandarizar los nombres de las ramas y las convenciones de commits para mantener un flujo de colaboración consistente.
- Mantener actualizadas las evidencias de cada historia de usuario en Jira y en el repositorio.